

Relatório do Projeto - Battle-CIn

Estruturas de Dados Orientadas a Objetos

Centro de Informática — Universidade Federal de Pernambuco

1. **Descrição do Projeto:** O Battle-CIn é um jogo auto-battle tático desenvolvido em C++17 com gráficos Raylib. Pode ser jogado por um ou dois jogadores. Cada jogador escolhe um treinador inspirado nos professores do CIn-UFPE e, após a escolha, tem acesso à equipe de heróis exclusiva daquele treinador. O jogador seleciona 3 desses heróis e define suas posições no campo de batalha. Em seguida, assiste ao combate automático num grid 8x8, podendo usar habilidades especiais durante a batalha e comprar itens entre os rounds.

2. Regras de negócio:

- 2.1. Cada treinador tem seus próprios heróis exclusivos e são inspirados em suas respectivas disciplinas;
- 2.2. Os dois jogadores não podem escolher o mesmo treinador;
- 2.3. Heróis se movem automaticamente em direção ao inimigo mais próximo;
- 2.4. O dano é calculado com base no AD do atacante e ARM do alvo;
- 2.5. O Mago ignora 50% da armadura no cálculo de dano;
- 2.6. O Assassino prioriza o inimigo com menos HP ao se mover;
- 2.7. Cada herói tem um timer de ataque baseado em sua velocidade (as_rate);
- 2.8. A habilidade do treinador só pode ser usada 1 vez por round;
- 2.9. Cada arquétipo tem uma condição específica para ativar o ultimate;
- 2.10. O primeiro a eliminar todos os heróis adversários vence o round;
- 2.11. O match é best-of-5 — primeiro a ter 3 vitórias de round vence;
- 2.12. Ao fim do match o resultado é salvo no banco de dados e impresso na tela.

3. Integrantes:

- 3.1. Cleyton Junior da Silva Cardoso — cjsc@cin.ufpe.br
- 3.2. Eliseu Cordeiro de Brito — ecb2@cin.ufpe.br
- 3.3. Matheus Victor Alves da Silva — mvas2@cin.ufpe.br
- 3.4. Hugo José Bento da Cunha — hjbc@cin.ufpe.br

4. Arquitetura do projeto

```

Servidor (src/server/)
├─ game.cpp      → Loop principal, fases, auto-battle, pathfinding, combate
├─ main.cpp      → Socket UDP, deserialização de pacotes, broadcast de snapshots

Cliente (src/client/)
├─ main.cpp      → Loop de jogo, input, interpolação visual, envio de pacotes
├─ renderer.cpp  → Renderização de grid, heróis, HUD, efeitos, targeting

Heróis (src/heroes/)
├─ hero.cpp      → Lógica base: stats, ataque, movimento, buffs, ultimate
├─ hero_factory.cpp → Criação polimórfica (TankHero, MageHero, etc.)

ED00 (include/)
├─ linked_list.h → Lista encadeada genérica + iterador
├─ graph.h       → Grafo do grid + BFS shortest path
├─ priority_queue.h → Max-heap para ordem de combate
├─ tournament_tree.h → Árvore binária de torneio
├─ database.h    → SQLite database para histórico e ranking

Protocolo (include/)
├─ protocol.h    → Pacotes UDP: InputPacket, SelectionPacket, TargetPacket, GameSnapshot

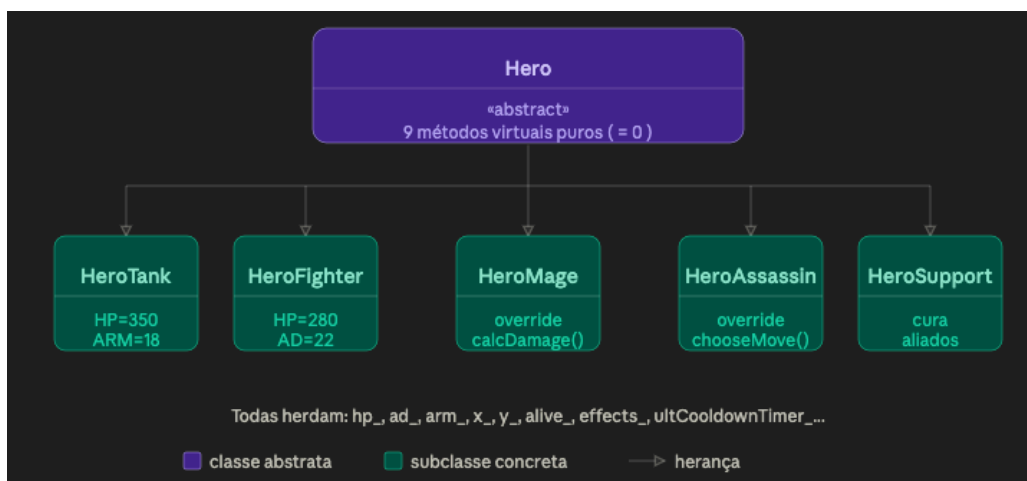
```

5. Requisitos:

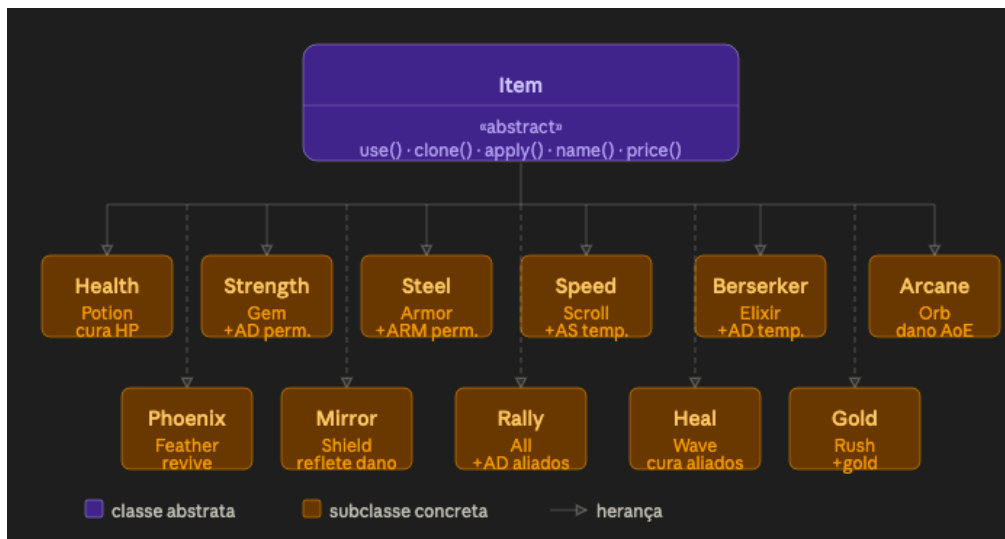
- 5.1. C++17
- 5.2. CMake 3.15+
- 5.3. Raylib (instalada no sistema ou via Conan)
- 5.4. SQLite3 (baixado automaticamente pelo Conan, ou instale manualmente:
`sudo apt install libsqlite3-dev`)
- 5.5. Conan (opcional, para gerenciamento de dependências)
- 5.6. Linux (desenvolvido e testado em ambiente Linux)

6. Diagramas:

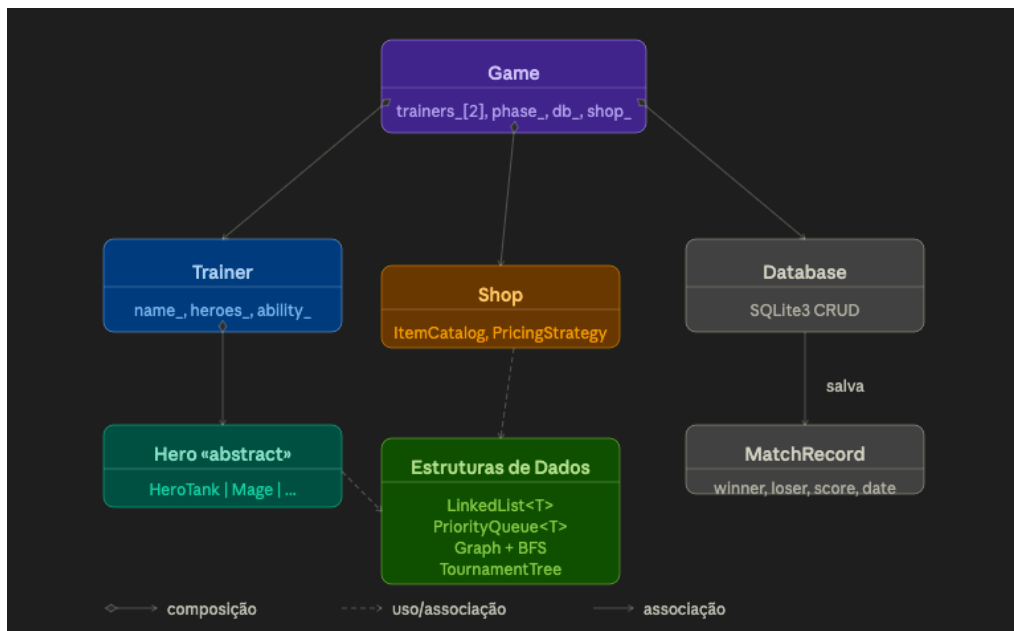
6.1. Classe Hero



6.2. Classe Item



6.3. Composição e associação



7. Classes e Objetos:

Classe	O que representa
Hero	Classe abstrata base para todas as unidades de combate
HeroTank	Subclasse concreta — herói tanque com HP e armadura altos
HeroFighter	Subclasse concreta — herói combatente com AD alto

HeroMage	Subclasse concreta — redefine calcDamage(), ignora 50% da armadura
HeroAssassin	Subclasse concreta — redefine chooseMove(), mira no inimigo com menos HP
HeroSupport	Subclasse concreta — cura aliados próximos no ultimate
Trainer	Gerencia uma equipe de 3 heróis e uma habilidade especial
Game	Controla toda a lógica do jogo: fases, combate, banco de dados
Database	Gerencia a persistência SQLite com as 4 operações CRUD
Shop	Loja de itens entre rodadas com catálogo e precificação dinâmica
Item	Classe abstrata base para os 11 tipos de itens da loja
HeroFactory	Factory Method — cria objetos Hero sem expor as classes concretas

Exemplo: HeroFactory

```

1  #include "../include/hero_factory.h"
2  #include "../include/hero_tank.h"
3  #include "../include/hero_fighter.h"
4  #include "../include/hero_mage.h"
5  #include "../include/hero_assassin.h"
6  #include "../include/hero_support.h"
7  #include "../include/protocol.h"
8  #include <cstdio>
9
10 std::unique_ptr<Hero> HeroFactory::create(uint8_t archetype, uint8_t ownerId)
11 {
12     switch (archetype) {
13         case ARCHETYPE_TANK:    return std::make_unique<HeroTank>(ownerId);
14         case ARCHETYPE_FIGHTER: return std::make_unique<HeroFighter>(ownerId);
15         case ARCHETYPE_MAGE:    return std::make_unique<HeroMage>(ownerId);
16         case ARCHETYPE_ASSASSIN: return std::make_unique<HeroAssassin>(ownerId);
17         case ARCHETYPE_SUPPORT: return std::make_unique<HeroSupport>(ownerId);
18         default:
19             printf("[HeroFactory] Archetype desconhecido %d\n", archetype);
20             return nullptr;
21     }
22 }
23
24 std::unique_ptr<Hero> HeroFactory::create(uint8_t archetype, uint8_t ownerId,
25                                           int hp, int ad, int arm, uint8_t heroDefIndex)
26 {
27     auto hero = create(archetype, ownerId);
28     if (hero) {
29         hero->setCustomStats(hp, ad, arm);
30         hero->setHeroDefIndex(heroDefIndex);
31         hero->resetStats(); // apply custom stats immediately
32     }
33     return hero;
34 }

```

8. **Herança:** O Battle-CIn utiliza herança em duas hierarquias distintas. Na hierarquia de heróis, a classe *Hero* é abstrata e serve como base para os cinco arquétipos do jogo: *Tank*, *Fighter*, *Mage*, *Assassin*, *Support*. Todos herdam automaticamente os atributos de combate como HP, dano, armadura e posição no grid, implementando apenas o que é específico de cada arquétipo. Na hierarquia de itens, a classe *Item* segue o mesmo princípio, com 11 subclasses concretas que implementam cada uma seu efeito específico como cura, aumento de atributos ou revive.
9. **Polimorfismo:** O polimorfismo é o mecanismo central do combate no Battle-CIn. O servidor chama os mesmos métodos para todos os heróis sem saber o tipo concreto de cada um. O comportamento muda automaticamente em tempo de execução dependendo do objeto real. Ao chamar *activateUltimate()*, por exemplo, o *Tank* ganha armadura, o *Fighter* ganha ataque, o *Mage* causa dano em área, o *Support* cura aliados e o *Assassin* teleporta para o inimigo mais fraco. Tudo com o mesmo trecho de código. O mesmo acontece com o cálculo de dano: o *HeroMage* redefine *calculateDamage()* para usar uma fórmula diferente, ignorando metade da armadura do alvo, enquanto os demais usam a fórmula padrão herdada de *Hero*. Isso foi possível graças aos métodos virtuais puros declarados em *Hero*, que obrigam cada subclasse a implementar seu próprio comportamento.
10. **Ponteiros:** No projeto, utilizamos o *unique_ptr*, um ponteiro inteligente da biblioteca padrão do C++ que gerencia a memória automaticamente, evitando vazamentos. Ele permite que um vetor de *Hero* guarde subclasses concretas como *HeroTank* e *HeroMage*, viabilizando o polimorfismo. Quando o *Trainer* é destruído, todos os heróis são liberados da memória automaticamente, sem necessidade de delete manual.
11. **Encapsulamento:** O encapsulamento no Battle-CIn protege o estado interno das classes, expondo apenas o necessário para o restante do código. Em *Hero*, os atributos de combate como HP, dano, armadura e posição foram declarados como *protected* em vez de *private*. Isso foi uma decisão intencional: como as subclasses *HeroTank*, *HeroMage* e as demais precisam acessar e modificar esses atributos diretamente para implementar seus comportamentos específicos, o *private* impediria esse acesso. O *protected* permite que as subclasses acessem os atributos herdados normalmente, enquanto ainda os mantém protegidos do restante do código externo. Para leitura pública, foram definidos getters *const* como *hp()*, *ad()* e *alive()*, garantindo que nenhuma classe externa modifique o estado interno diretamente. Na classe *Database*, toda a lógica SQLite foi declarada como *private*, expondo apenas os métodos do CRUD para o restante do sistema.
12. **CRUD:** O Battle-CIn utiliza um banco de dados SQLite3 para persistir o histórico de partidas e os dados do jogo. Toda a comunicação com o banco é encapsulada na classe *Database*, que expõe apenas as quatro operações do CRUD: Create, Read, Update e Delete.
O **Create** acontece em dois momentos: na primeira execução do servidor, quando o banco é criado automaticamente e populado com os dados de heróis, treinadores e itens da loja; e ao fim de cada round, quando o resultado da partida é salvo automaticamente no histórico.

O **Read** é usado para carregar os dados do jogo ao iniciar o servidor e para exibir o ranking dos treinadores na tela de fim de match, calculado a partir do histórico de partidas salvo.

O **Update** permite atualizar os atributos base de um herói diretamente no banco, como HP, dano e armadura, sem precisar recompilar o projeto.

O **Delete** zera o histórico de partidas, resetando o ranking de todos os treinadores.

13. Divisão de Trabalho:

- 13.1. **Matheus Victor** foi responsável pela base inicial do projeto: estruturou a arquitetura cliente-servidor UDP, definiu o protocolo de comunicação entre cliente e servidor, implementou a hierarquia de heróis com as classes abstratas Hero e Item.
- 13.2. **Eliseu Brito** implementou o sistema de loja entre rodadas com o catálogo de itens, a estratégia de precificação dinâmica e os efeitos de cada item. Melhorou significativamente a interface gráfica, implementou as sprites dos heróis e treinadores, adicionou animações de ataque e enriqueceu os heróis com novos atributos e comportamentos.
- 13.3. **Cleyton Cardoso** implementou o banco de dados SQLite com o CRUD completo: histórico de partidas, ranking de treinadores e dados dos heróis. Também desenvolveu a tela de ranking exibida ao fim de cada match e gerou as imagens dos treinadores.
- 13.4. **Hugo José** definiu o sistema de exclusividade de heróis por treinador, garantindo que cada professor tenha sua própria equipe. Criou os heróis do jogo com seus nomes, descrições e imagens, além de balancear os atributos de cada um.